ДИСЦИПЛИНА	Технологии разработки серверных приложений
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Текущий контроль
ПРЕПОДАВАТЕЛЬ	Дворецкий Артур Геннадьевич
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Ссылка на материал:

https://github.com/dv0retsky/fastapi-tutorial/blob/main/FAPI_KR4/FAPI_KR4.md

---

Контрольная работа №4 🎧

🦄 Как сдавать работу

1. Выполните задания в **отдельном репозитории** (рекомендуется: один репозиторий на одну контрольную работу).
2. Загрузите исходный код в репозиторий (`commit` + `push`).
3. В **СДО** прикрепите ответ в виде **ссылки на репозиторий с решением**.

> **Рекомендации к оформлению репозитория:**
>
> - Проект должен **запускаться** и быть готов к проверке.
> - Добавьте зависимости: `requirements.txt` или `pyproject.toml`.
> - В `README.md` кратко опишите:
> - как установить зависимости и запустить приложение;
> - как проверить основную функциональность проекта;
> - какие команды использовать для тестирования ключевых сценариев.
> - Если в проекте используются переменные окружения, добавьте файл **`.env.example`** и **не публикуйте реальные секреты**.
> - Файл `.env` должен быть добавлен в `.gitignore`.
> - Если в работе предусмотрены тесты, миграции, документация API или дополнительные сервисы, убедитесь, что проверяющий может запустить их без доработок.

🌟 Задание 9.1

Для выполнения этой задачи программирования вы настроите Alembic для миграции базы данных в приложении FastAPI и создадите простую миграцию для изменения схемы базы данных.

Требования:

1. **Установите** Alembic и необходимый драйвер базы данных для выбранной вами базы данных (например, SQLite, PostgreSQL, MySQL).
2. **Настройте** Alembic, инициализировав его в вашем проекте FastAPI и настроив в соответствии с настройками вашей базы данных.
3. Создайте **модель данных** (модель SQLAlchemy), представляющую ресурс (например, объект `"Product"`) с такими полями, как `"id"`, `"title"`, `"price"` и `"count"`.
4. Сгенерируйте **начальный сценарий** миграции с помощью Alembic, который создает новую таблицу для объекта `"Product"`.
5. **Примените** первоначальную миграцию к базе данных и убедитесь, что таблица `"Product"` создана. После этого добавьте две записи в таблицу.
6. **Измените** модель данных, добавив новое поле, например `"description"`, к сущности `"Product"`. `description` должен быть `not null`.
7. Сгенерируйте **новый сценарий** миграции с помощью Alembic, который отражает изменения в сущности `"Product"`.
8. **Примените новую миграцию** к базе данных и убедитесь, что таблица `"Product"` теперь содержит поле `"description"`.

> **Примечание:** Миграция базы данных является важным аспектом управления изменениями схемы базы данных в реальных приложениях. Эта задача предоставляет упрощенный сценарий, и в производственной среде вы должны следовать дополнительным рекомендациям, обрабатывать миграцию данных и учитывать возможные сценарии потери данных. **Если вам привычнее работать в другим технологическим стеком, то изучите возможность выполнить задание (миграцию БД) подходящим для вас способом.**

🌟 Задание 10.1

Для этой задачи программирования вам необходимо реализовать пользовательскую обработку ошибок в приложении FastAPI. Ваше приложение должно включать

пользовательские классы исключений, обработчики исключений и модели реагирования на ошибки.

Требования:

1. Создайте приложение FastAPI и настройте пользовательские **классы** исключений по крайней мере для двух конкретных исключений (например, `CustomExceptionA` и `CustomExceptionB`). Каждое пользовательское исключение должно иметь уникальный код состояния и пользовательское сообщение об ошибке.
2. Зарегистрируйте пользовательские **обработчики** исключений для двух пользовательских исключений. Обработчики исключений должны возвращать ответы об ошибках с соответствующими кодами состояния и сообщениями об ошибках.
3. Определите **модели** реагирования на ошибки, используя модели Pydantic, чтобы обеспечить согласованное форматирование ответов на ошибки во всем приложении.
4. Реализуйте по крайней мере **две конечные точки API**, которые вызывают пользовательские исключения в определенных сценариях. Например, одна конечная точка может вызвать `CustomExceptionA`, когда не выполняется определенное условие, а другая конечная точка может вызвать `CustomExceptionB`, когда ресурс не найден.
5. **Протестируйте** свою пользовательскую обработку ошибок, отправив запросы к конечным точкам API, которые вызывают пользовательские исключения. Убедитесь, что ответы об ошибках содержат правильные коды состояния и сообщения об ошибках.

> **Примечание:** Для этой задачи вы можете использовать простые инструкции печати для регистрации ошибок. В реальном сценарии вы обычно использовали бы библиотеку ведения журнала для эффективной обработки регистрации ошибок.

🌟 Задание 10.2

Для этой задачи программирования вам необходимо реализовать **проверку данных запроса и пользовательскую обработку ошибок проверки** в приложении FastAPI.

Требования:

1. Создайте приложение FastAPI по крайней мере **с одной конечной точкой, которая принимает полезную нагрузку** JSON, представляющую данные пользователя (например, `"name"`, `"age"`, `"e-mail"` и др. согласно примеру ниже, либо можете использовать свой пример).

2. Определите **модель Pydantic для пользовательских данных**, чтобы выполнить проверку входящей полезной нагрузки JSON.

```
class User(BaseModel):
    username: str
    age: conint(gt=18)
    email: EmailStr
    password: constr(min_length=8, max_length=16)
    phone: Optional[str] = 'Unknown'
```

Немного прокомментируем, чтобы вы знали больше, чем мы рассказывали ранее:

- В `username` - проверка на строку, ничего необычного;
- `conint` - это `int` с ограничениями (от "add constraints");
- `gt=18` - greater than - больше чем; прочитайте про другие сокращения в FastAPI;
- `EmailStr` - проверяет, удовлетворяет ли строка требованиям к емейлу (либо можете написать сами регулярку);
- `constr` - аналогично предыдущему, но строка с ограничениями (ограничения по длине - заданы минимальная и максимальная длины);
- `Optional[str]` - переменная имеет тип str, но это является «необязательным». И также указали значение по умолчанию.

3. Реализуйте **пользовательскую обработку ошибок** проверки с помощью декоратора `@app.exception_handler`, чтобы предоставлять информативные ответы об ошибках при сбоях проверки.
4. **Протестируйте обработку ошибок** пользовательской проверки, отправив запросы с неверными пользовательскими данными в конечную точку и убедившись, что ответы об ошибках содержат соответствующие коды состояния и сообщения об ошибках.

Примечание: Вы можете использовать `fastapi.testclient.TestClient` для выполнения тестовых запросов к вашему приложению FastAPI и проверки ответов (про него мы расскажем в следующем блоке).

🌟 Задание 11.1

Для этой задачи программирования вам необходимо написать модульные тесты для приложения FastAPI, которое включает в себя несколько конечных точек и функциональных возможностей.

Требования:

1. Настройте приложение FastAPI как минимум с тремя конечными точками API (например, одна для регистрации пользователя, одна для извлечения данных и одна для удаления данных). В качестве хранилища допускается использовать массив/словарь.
2. Напишите модульные тесты для каждой конечной точки с помощью `pytest`. Протестируйте различные сценарии и крайние случаи, чтобы обеспечить всесторонний охват тестированием.
3. Для каждого тестового примера используйте `TestClient` для отправки HTTP-запросов на соответствующую конечную точку и проверки кода состояния ответа и данных с помощью утверждений (`assert`).
4. Организуйте связанные тестовые примеры в отдельные тестовые функции или тестовые классы для удобства сопровождения и чтения.
5. Убедитесь, что модульные тесты прошли успешно, указывая на то, что конечные точки API и функциональные возможности работают должным образом.

После выполнения задачи у вас должен быть набор модульных тестов, которые тщательно проверяют функциональность вашего приложения FastAPI. Выполнение тестов с помощью `pytest` должно привести к прохождению всех тестовых примеров, что указывает на надежность вашего приложения и отсутствие критических ошибок.

> **Примечание:** Вы можете (но не обязательно) использовать фреймворк `pytest` и плагин `pytest-asyncio` для написания асинхронных модульных тестов в FastAPI.

🌟 Задание 11.2

Предположим, что в предыдущем задании у вас было простое приложение FastAPI с тремя конечными точками и хранилищем в памяти (обычный словарь). Если у вас есть своё решение — используйте его. Если нет, можно опереться на этот минимальный пример:

```
# my_project/main.py
from fastapi import FastAPI, HTTPException, Response
from pydantic import BaseModel
from itertools import count
from threading import Lock

app = FastAPI()

db: dict[int, dict] = {}
```

```python
_id_seq = count(start=1)
_id_lock = Lock()


def next_user_id() -> int:
    with _id_lock:
        return next(_id_seq)


class UserIn(BaseModel):
    username: str
    age: int


class UserOut(BaseModel):
    id: int
    username: str
    age: int


@app.post("/users", response_model=UserOut, status_code=201)
def create_user(user: UserIn):
    user_id = next_user_id()
    db[user_id] = user.model_dump()
    return {"id": user_id, **db[user_id]}


@app.get("/users/{user_id}", response_model=UserOut)
def get_user(user_id: int):
    if user_id not in db:
        raise HTTPException(status_code=404, detail="User not found")
    return {"id": user_id, **db[user_id]}


@app.delete("/users/{user_id}", status_code=204)
def delete_user(user_id: int):
    if db.pop(user_id, None) is None:
        raise HTTPException(status_code=404, detail="User not found")
    return Response(status_code=204)
```

В этом задании нужно написать **асинхронные модульные тесты** к этому приложению (можете использовать свой код или минимальный вариант выше), задействовав `pytest-asyncio`, `httpx.AsyncClient` (через `ASGITransport`) и библиотеку **Faker** (`pip install Faker`) для генерации данных.

Что нужно сделать

1. Подготовка окружения

Установите зависимости: `pytest`, `pytest-asyncio`, `httpx`, `Faker`. Настройте структуру проекта с папкой tests/ и файлом настроек `Pytest` при необходимости.

2. Асинхронные тесты для всех трёх эндпоинтов

Напишите тесты для сценариев:

- создание пользователя (`201`) и валидация структуры ответа;
- получение существующего пользователя (`200`);
- попытка получить несуществующего пользователя (`404`);
- удаление существующего пользователя (`204`);
- повторное удаление того же пользователя (`404`).

3. HTTP-клиент без реального сервера

Для обращений к приложению используйте `httpx.AsyncClient` с `ASGITransport`, чтобы тесты ходили в приложение напрямую, минуя запуск `Uvicorn`.

4. Генерация данных через Faker

Используйте `Faker` (и/или `pytest`-фикстуру `faker`) для реалистичных данных: имя пользователя, возраст и т.п. Проверьте как валидные, так и граничные значения.

5. Изоляция состояния

Обеспечьте «чистое» состояние между тестами (очистка вашего in-memory словаря до/после каждого кейса), чтобы тесты не влияли друг на друга. При желании сгруппируйте кейсы в тестовый класс.

6. Запуск и проверка

Запустите `pytest` и убедитесь, что все тесты проходят.

Критерии приёмки

- Тесты **асинхронные** (используется `pytest-asyncio`) и выполняются через `httpx.AsyncClient` + `ASGITransport`.
- В тестах применяется **Faker** для генерации входных данных.
- Покрыты успешные и базовые ошибочные сценарии для всех трёх эндпоинтов.
- Состояние хранилища изолировано между тестами.
- Все тесты завершаются статусом **passed**.

Полезные материалы

- FastAPI — официальное руководство по **async-тестам** (пример с `httpx.AsyncClient` и `ASGITransport`). (fastapi.tiangolo.com)
- `pytest-asyncio` — документация (основы и маркер для асинхронных тестов). (pytest-asyncio, GitHub)
- HTTPX — **Async Support** и **ASGI Transport** (клиент для асинхронных тестов, вызовы приложения напрямую). (python-httpx.org)
- **Faker** — общая документация и фикстура `faker` для Pytest. (faker.readthedocs.io)
- **Хабр**: обзоры по асинхронному тестированию и Pytest (на русском). (Habr)
- **Stack Overflow:** практические Q&A по асинхронным тестам FastAPI с Pytest/HTTPX. (Stack Overflow, Stack Overflow на русском)

Made with ❤️ by **dv0retsky**